# 实验一：词法分析与语法分析

参与人员：

- 231220075 黄睿智
- 231220077 白子敬

## 1. 实验任务

- 必做：搭建词法分析器（ `lexical.l` ）和语法分析器（ `syntax.y` ），完成词法和语法分析。
- 选做任务：2.1 识别八进制和十六进制数。
- 我们实际三个选做任务都做了，因为担心可能会影响后续实验的进行，或者迟早都要做。

## 2. 代码文件说明

- `lexical.l`
  - 功能：
    - 识别关键字 `int` `float` `struct` `return` `if` `else` `while` 、标识符、整型（十进制/八进制/十六进制）、浮点数（包括科学计数）、运算符、分隔符。
    - 注释（ `//` 行注释、 `/* ... */` 块注释）和空白符跳过。
    - 非法数值（ `BAD_OCT/BAD_HEX/BAD_EXP` ）与非法字符通过 `report_lexical_error` 输出“Error type A”并赋值 `has_lexical_error` 。
  - 思路：
    - 先用宏定义统一描述词法单元（如 `DIGIT` 、 `ID` 、 `FLOAT_EXP` ），再按“关键字/运算符/错误模式/正常数字/标识符”的顺序组织规则，保证匹配优先级。
    - 使用 `%x COMMENT` 独立注释状态，避免 `/* ... */` 与普通 `/` 运算符规则相互干扰。
    - 使用 `YY_USER_ACTION` 与 `yylineno` 同步位置信息，把 token 的位置信息直接传给语法分析器，便于后续统一报错行号。
    - 非法数字在报错后返回占位 token（如 `INT/FLOAT` ），这样可以继续扫描后续内容，尽可能一次性给出更多词法错误，而不是在第一个错误处提前中断。
- `syntax.y`
  - 功能：
    - 定义 `TreeNode` 语法树结点结构与 `new_terminal/new_nonterminal` 工具函数。
    - 语法树根 `syntax_root` 由 `Program -> ExtDefList` 产生。

        - 规则包括：类型定义、变量声明、函数定义、语句（表达式、复合语句、if-else、while、return）、表达式运算优先级（赋值、逻辑、关系算术、数组/结构体访问、函数调用等）。
        - 错误恢复： `error` 产生式，输出 `report_syntax_error` （“Error type B”）并使用 `yyerrok` 恢复解析。
    - 实现思路：
        - 采用“语法识别 + 同步建树”的方式：每个产生式在归约时调用 `NODE/TOKEN` 生成结点。
        - AST 结构设计为“child-sibling”链式多叉树，直接表达变长语法结构（如参数列表、语句列表），也方便递归打印与递归释放。
        - 通过 `%left/%right/%nonassoc` 明确运算符优先级与结合性。
        - 错误处理分层：词法阶段记录 A 类错误；语法阶段通过 `error` 产生式做局部恢复，并对同一行重复错误做抑制，避免因为词法报错抛弃该非法词组而产生的重复报错。
          eg: `int i = 09;` 可能会同时报非法八进制数和 `syntax error` 。
- `main.c`
    - 负责逐文件解析调用 ( `parse_file` )。
    - 调用 `reset_parser_state` ，重置 `yylineno` 和状态。
    - 仅当无词法/语法错误时输出语法树 `print_tree(syntax_root, 0)` 。

# 3. 编译/运行方式

- 进入目录： `cd /Lab/Code` 。
- 直接运行： `make` ，生成 `paser` 可执行文件。
- 运行测试：
    - `./parser ../Test/test{1/2/3/4}.cmm`
- 清理： `make clean` 。

# 4. 测试验证

- test1: 行注释和块注释。
- test2: 指数形浮点数。
- test3: 非法八进制和十六进制数。
- test4: 常规测试。